EE450 Socket Programming Project (Spring 2026)

Due Date: Apr 28, 2026

Problem Statement

You are a third-party security and network engineer contracted to design and implement a healthcare application for a hospital. The system must comply with the Health Insurance Portability and Accountability Act (HIPAA), requiring the protection of personally identifiable information (PII) and protected health information (PHI).

The application will support both doctors and patients and will include user authentication, the ability to schedule and manage medical appointments, and the creation and updating of prescription records.

To meet HIPAA security requirements, the system must implement secure authentication mechanisms, including cryptographic hashing of passwords and appropriate cryptographic protection for user identifiers. The application must ensure confidentiality, integrity, and access control while maintaining reliability and availability for authorized users.

This project will require three backend servers: **Authentication Server**, **Appointment Server**, and **Prescription Server**. Users will perform various operations through a client interface, which will dispatch them to the corresponding servers via the hospital server.

- **Client:** This interface will allow users to perform actions based on their role
    - **Doctors:** Can log in, view scheduled appointments, write prescriptions to clients (assuming that their appointment occurred) based on illness, look up prescriptions for patients.
    - **Patients:** Can log in, look up the availability of doctors, create an appointment with a doctor, view and cancer their appointment, and view their status of their prescription.
- **Hospital Server:** Handles all actions by dispatching requests to appropriate backend servers and by handling hospital-related tasks, such as granting doctor or patient access.
- **Backend Servers:** *Authentication* server, *Appointment* server, and *Prescription* server are used for authentication, scheduling and viewing appointments, and storing prescriptions for clients, respectively.

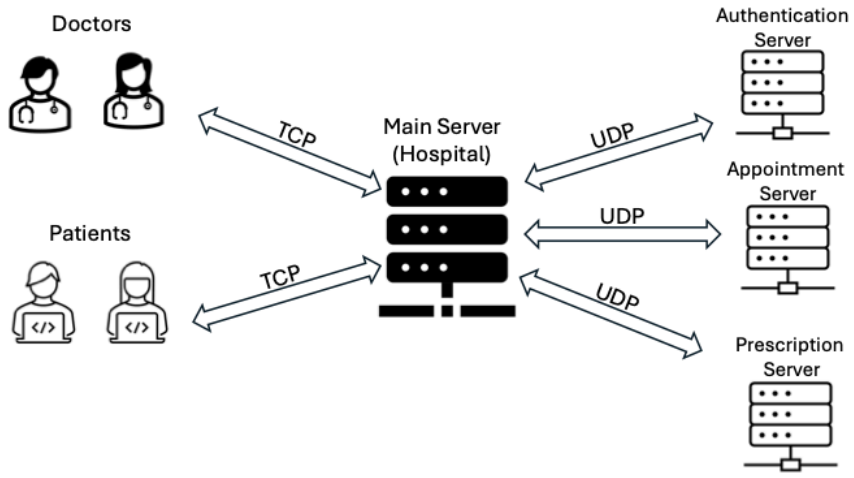

Figure 1. Illustration of the network

Source Code Files

Your implementation should include the source code files described below, for each component of the system

1. Client:
   You must name your code file: ***client***.*c* or ***client***.*cc* or ***client***.*cpp* or ***client***.*py* (all small letters), and the header file (if you have one; it is not mandatory) must be called ***user_server***.*h* (all lowercase).

2. Hospital Server:
   You must name your code file: ***hospital_server***.*c* or ***hospital_server***.*cc* **OR** ***hospital_server***.*cpp* (all small letters). Also, you must include the corresponding header file (if you have one; it is not mandatory): hospital_server.h (all lowercase).

3. Backend Servers: You are required to create three distinct files, each using one of the following naming conventions: *[placeholder]*_server.c or *[placeholder]*_server.cc **OR** *[placeholder]*_server.cpp. The filename must utilize these formats, substituting "*placeholder*" with the specific server identifier (either "***authentication***", "***prescription***", or "***appointment***") to reflect the server it represents, resulting in filenames like *authentication_server*.c or ***authentication_server***.cc, ***authentication_server***.cpp, ***authentication_server***.py (note that the name should be entirely in lowercase except for the letter replacing "placeholder"). If available, you should also include a corresponding header file named *[placeholder]*_server.h, adhering to the same naming rule for the "*placeholder*" replacement. This ensures a clear, organized naming structure for your code and its associated header file, if any.

Note: You are not allowed to use one executable for all four servers (i.e., a "fork" based implementation)

Files

1. "***users.txt***":
   Located in ***authentication_server*** (Authentication server), which manages user access. This file is formatted with two **SHA-256 HASHED** columns: ***usernames*** and ***passwords*** (in this corresponding order). A space separates these columns " ''. Each patient's data is stored on a separate, new line. The users.txt file is used for authentication purposes.

2. "**hospital.txt**":
   Located in ***hospital_server*** (Hospital server), which manages doctor access and the pharmacy. This file is formatted like so:
   [*Doctors]*
   <doctor_name_1> <hashed_doctor_name>
   <doctor_name_2> <hashed_doctor_name>
   <doctor_name_3> <hashed_doctor_name>

   [Treatments]
   <illness_1> <treatment_1>
   <illness_2> <treatment_2>
   …

3. "**appointments.txt**"
   Located in ***appointment_server*** (Appointment server), which manages doctor appointments and their patients. This file is formatted like so:
   doctor_name_1
   <start_time_1> <hashed_patient_1> <illnessID_1>
   <start_time_2> <hashed_patient_2> <illnessID_2>
   …

   doctor_name_2
   <start_time_1> <hashed_patient_1> <illnessID_1>
   <start_time_2> <hashed_patient_2> <illnessID_2>
   …
   Each doctor has 8 time slots, starting from 9:00 and ending at 17:00 (not inclusive). To determine if the time slot is taken, the time slot must proceed with the hash of the patient username and illness. If only the timeslot is on the line, it means the Doctor is available at this time block.

4. "**prescriptions.txt**"
   Located in ***prescription_server*** (prescription server), which manages the treatment prescribed by doctors to patients. This file is formatted with four columns:

   <doctor_name_1> <*hashed_patient*> <*treatment*> <*frequency*>

A space separates these columns “ ‘’. Each prescription is stored on a separate, new line. The **prescriptions***.txt* file is used for prescription purposes.

Problem Definition Details

Phase 1A:

Please refer to the “Process Flow” section to start your programs in the order of the ***hospital _server***, ***authentication_server***, **appointment_server**, **prescription_server,** and at least two Clients (one **Patient** and one **Doctor**).

Your programs must start in this order. Each of the servers and the clients have boot-up messages that must be printed on the screen. Please refer to the on-screen messages section for further information. When three backend servers (***authentication_server***, **appointment_server,** and **prescription_server**) are up and running.

You can choose any data structure that accommodates your needs. You should print the correct on-screen messages for the hospital server and the backend servers, indicating the success of the operations mentioned in Phase 2, as described in the “ON-SCREEN messages" section. After the servers are booted up, the client will be started. Once the client boots up and the initial boot-up messages are printed, the client waits for the user to check the authentication.

Phase 1B:
Authentication:
Command format: ./client <username> <password>
In this phase, the client will be asked to enter the username and password on the terminal. To authenticate, the user will run the command as described at the beginning of this section.

To provide confidentiality and security, you must make sure that both of these arguments are hashed using the SHA-256 algorithm. Hashing is a one-way process that converts input data into a fixed-length value that cannot be reversed to obtain the original input. SHA-256 is a cryptographic hash function that produces a 256-bit output and is widely used in modern systems. In HIPAA-compliant healthcare applications, hashing is commonly used to protect sensitive information, such as passwords, and to pseudonymize patient identifiers. By storing hashed identifiers instead of plaintext patient names, backend systems reduce exposure of protected health information while still allowing consistent identification and access control through trusted services.

All users must provide their credentials to the client. The client must ensure that both the username and password are hashed before submitting a request to the hospital server, which will then request information from the authentication server. The authentication server has the users.txt file and checks whether an entry in it matches the hash of both the username and password. If a line containing the username hash and the hashed password is found, the authentication server responds with success. The hospital server must then determine if the user is a doctor by checking in hospital.txt to see if the given hash matches a doctor in the

system. If not, it will grant the user patient access. A response from the authentication server will then be sent back to the client.

If the credentials provided do not fall under any of the previous cases, the authentication server, hospital server, and client must return an error message (as detailed in the ON SCREEN messages section), and the process must end. If a client wants to start over, they need to run the command line again.

For those using C or C++, you have to download two files (sha256.h and sha256.c) that implement SHA-256 from this GitHub repository: https://github.com/LekKit/sha256

- ➔ For those using C++, you need to change the extension of SHA256.c to SHA256.cpp to compile it using C++ and avoid a warning
- ➔ Include either sha256.c or sha256.cpp, depending on your implementation, with your program.

You will need to call sha256_easy_hash_hex(data, size, output), where data is the input string (or character array) to be hashed, size is the length of that input data, and output is the buffer where the resulting SHA-256 hash in hexadecimal format will be stored. Ensure that the output buffer is large enough to hold the full 64-character hexadecimal hash, plus a null terminator if required.

For those using Python, you can use the built-in library that provides SHA256 functionalities. Note that this removes any leading or trailing whitespaces:

```
def sha256_hash(text: str) -> str:
  text = text.strip()
  return hashlib.sha256(text.encode('utf-8')).hexdigest()
```

Double check that your program is working correctly by testing these examples (you can try hashing by pasting text into this website https://emn178.github.io/online-tools/sha256.html) :

Example	Original Text	Hash
#1	Welcome to EE450!	2579794c502c6c6c9f48da90e606188cee7609e1889a2af8caa2eee00bb1fd9d
#2	199xyz@$	7f7a9dc72b1ddaa311477624f64cbfc1b53caff50dcf16469d7e36fe155a4228
#3	0.27#&	2f8aa685fdfd3d32a7f7efc7681cc4a816e39381334f05cc3b132dd5ca83c00e

Warning:

- Because hashing is case-sensitive, any change in letter capitalization will result in a different hash value. Usernames and passwords must be entered exactly as originally specified.

Phase 2:

Client Commands:

Command format: **lookup**

This command retrieves the list of doctors from the hospital.txt file on the hospital server.

Command format: **lookup** <doctor>

This command retrieves the doctor's availability from the appointment server, routed through the hospital server.

Command format: **schedule** <doctor> <block period> <Illness>

This command will schedule a time block with a doctor. This will route the message through the hospital server and to the appointment server. This request will only succeed if the two conditions are met:

- The requested time is within 9:00-16:00 (8 time blocks are available) and must be in the format HH:MM.
- The time slot is available

Display a success message if the appointment server has successfully allocated a time slot for that patient, and display a failure if the conditions were not met.

Command format: **view_appointment**

Display the appointment that the person has allocated.

Command format: **cancel**

This command will cancel the patient's appointment. A response will be routed from the appointment server through the hospital server. Due to using txt files, you must iterate through the whole file to find if there is an instance of the patient. If there is, remove their information from the system. Note that you should still keep the timeslot in the file.

Doctor Commands:

Command format: **view_appointments**

Display the list of time blocks the doctor has been scheduled for.

Phase 3:

Doctor Commands:

Command format: **prescribe** <patient_username> <length_period>

Assume that after the appointment, the doctor has saved the patient's username in a notepad and immediately prescribes the patient the treatment listed in hospital.txt. The process goes as follows:

- The doctor's client sends the prescription request to the hospital server
- The hospital server fetches the patient's illness from the appointment server.

- ➢ The appointment server searches the appointments.txt file for the entry containing the patient's hashed identifier. Once the matching time block is found, the server retrieves the associated illness value. The server then removes the hashed patient identifier and illness from that time block, leaving only the original time entry to indicate that the slot is now available in appointment.txt. The entry itself must remain in the file. The illness value is returned to the hospital server for further processing.
- ➢ The hospital server looks at the treatment found under hospital.txt, and requests the prescription server to save the treatment and frequency for the patient given by the doctor.

After this process, the servers should display the messages for this process. The entries for frequency are as follows: "*None*", "*Daily*", "*Bi-daily*", "*Weekly*".

Command format: **view_prescription** <patient_username>
This command returns back the prescription that was saved in prescription.txt to the doctor.

Client Commands:
Command format: **view_prescription**
The view prescription command allows an authenticated patient to retrieve their assigned prescription. When this command is issued, the client sends a request to the hospital server, which forwards the patient's hashed identifier to the prescription server. The prescription server searches for a matching record associated with that patient. If a prescription exists, the system displays the prescribing doctor's name, the treatment name, the quantity, and the frequency. If the frequency field is recorded as "None", the system displays the message: *"You were not prescribed a treatment from <doctor> ."* This ensures that patients are clearly informed when no active prescription has been assigned to them after their appointment evaluation.

Miscellaneous commands:
Command Format: **help**
Displays the available commands

Command format: **quit**
Ends the connection between the client and server.

Required Port Number Allocation

The ports to be used by the client and the servers for the exercise are specified in the following table:

Table 2. Static and Dynamic assignments for TCP and UDP ports.		
Process	**Dynamic Ports**	**Static Ports**
authentication_server	-	1 UDP , 21000+xxx
prescription_server	-	1 UDP , 22000+xxx
appointment_server	-	1 UDP , 23000+xxx
hospital_server	-	1 UDP , 25000+xxx, 1 TCP with client, 26000+xxx
client	2 TCPs	-

NOTE: xxx is the last 3 digits of your USC ID. For example, if the last 3 digits of your USC ID are “319”, you should use the port: **21000+319 = 21319** for the Backend-Server (A). **It is NOT going to be 21000319.**

On-Screen Messages

Note: For displaying the hashed value, display the last 5 hex char, which will now be referenced as **hash_suffix**.
Ex: SHA256(Welcome to EE450!) = 2579794c502c6c6c9f48da90e606188cee7609e1889a2af8caa2eee00bb1fd9d
Then hash_suffix would be **1fd9d**

Table 3. Authentication Server	
Event	**On-Screen Message**
Boot-Up	**“**Authentication Server is up and running using UDP on port <port number>.”
Upon Receiving an Authentication Request	“Authentication Server has received an authentication request for a user with hash suffix: <hash_suffix>.”
If User Credentials Are Valid	“Authentication succeeded for a user with hash suffix: <hash_suffix>.”

Table 3. Authentication Server	
If user credentials are invalid	“Authentication failed for a user with hash suffix: <hash_suffix>.”
Upon sending the authentication result to the Hospital Server	“The Authentication Server has sent the authentication result to the Hospital Server.”

Table 4. Appointment Server	
Event	**On-Screen Message**
Boot-Up	“Appointment Server is up and running using UDP on port <port number>.”
lookup request	
Upon receiving appointment scheduling request	“Appointment Server has received a doctor availability request.”
If the doctor does not match	“<doctor_username> was not found in the system.”
After doctor entry is found	“<doctor_username> was found in the system.”
schedule request	
Upon receiving appointment scheduling request	“Appointment scheduling request received (time: <time_block>, doctor: <doctor_name>, patient hash suffix: <hash_suffix>, illness: <illness>).”
If the appointment Is successfully scheduled	“Appointment has been scheduled successfully for user <hash_suffix> with <doctor_username>.”
If the requested time slot is not available	“The requested appointment time is not available.”
view_appointment request	
Upon receiving to view appointment request	“Appointment Server has received a view appointment command for the user with hash suffix <hash_suffix>.“
Appointment was found	“Returning details regarding the appointment for the user with hash suffix <hash_suffix>.”
No appointment was found	“The user with hash suffix <hash_suffix> has no appointment in the system.”

Table 4. Appointment Server	
cancel request	
Upon receiving Cancel appointment request	“Appointment Server has received a cancel appointment command for the user with hash suffix: <hash_suffix>.
Appointment was found and canceled	“Successfully canceled appointment.”
Appointment was not found	“Error: Failed to find appointment.”
view_appointments request (Doctor)	
Upon Receiving To View Appointments Request for doctor	“Appointment Server has received a request to view appointments scheduled for <doctor_username>.”
If no appointments are present	“No appointments have been made for <doctor_username>.”
Appointment(s) were found	“Returning the scheduled appointments for <doctor_username>.”
prescribe request (Doctor)	
Upon receiving request for patient's illness	“Appointment Server has received a request from Hospital Server regarding information about a user with hash suffix <hash_suffix> from <doctor_username>.”
After finding the entry of the requested patient	“Sending back the requested information to the Hospital server.”
After removing the patient from the doctor’s schedule	“Successfully removed <hash_suffix> appointment slot, <time_block> is now free to be scheduled for tomorrow.”

Table 5. Prescription Server	
Event	**On-Screen Message**
Boot-up	“Prescription Server is up and running using UDP on port <port number>.”
prescribe request (Doctor)	
Upon receiving a prescription request	“Prescription Server has received a request from <doctor_username> to prescribe the user with hash suffix <hash_suffix>.”

Table 5. Prescription Server	
After saving the patient's prescription to the server	"Successfully saved the prescription details for user with hash suffix: <hash_suffix>."
view_prescription request (Doctor and Patient)	
Upon receiving prescription request	"The prescription server has received a request to view the prescription for the user with hash suffix: <hash_suffix>."
No prescription exists or frequency entry is "None"	"There are no current prescriptions for this user."
Prescription exists	"A prescription exists for this user."

Table 6. Hospital Server	
Event	**On-Screen Message**
Boot-Up	"Hospital Server is up and running using UDP on port <port number>."
Authentication flow	
Upon receiving the authentication request from the client	"Hospital Server received an authentication request from a user with hash suffix <hash_suffix>."
Upon sending the authentication request to the Authentication Server	"Hospital Server has sent an authentication request to the Authentication Server."
Upon receiving the authentication response from the Authentication Server	"Hospital server has received the response from the authentication server using UDP over port <hospital Server UDP port number>."
If the response message grants access to the system	"User with a hash suffix <hash_suffix> has been granted access to the system. Determining the access of the user."
If the hash matches that of a doctor in hospital.txt	"User with hash suffix <hash_suffix> will be granted doctor access."
If the hash does not match that of a doctor in hospital.txt	"User with hash <hash_suffix> will be granted patient access."
Upon sending the authentication response to the client	"Hospital Server has sent the response from Authentication Server to the client using TCP over port <hospital server TCP port number>."

Table 6. Hospital Server	
lookup request	
Upon receiving the lookup request for doctors from the client	“Hospital Server received a lookup request from a user with a hash suffix <hash_suffix> over port <hospital server TCP port number>.”
After sending the lookup response to the client	“Hospital Server has sent the doctor lookup to the client.”
lookup <doctor_username> request	
Upon receiving the lookup request from the client to get the doctor’s availability	“Hospital Server has received a lookup request from a user with hash suffix <hash_suffix> to lookup <doctor_username> availability using TCP over port <hospital server TCP port number>.”
After forwarding the lookup request to the appointment server	“Hospital Server sent the doctor lookup request to the Appointment server.”
After receiving the response from Appointment Server	“Hospital Server has received the response from Appointment Server using UDP over port <hospital Server UDP port number>.”
After forwarding the response to the client	“The Hospital Server has sent the response to the client.”
schedule request	
Upon receiving the schedule request from the client to get a doctor’s availability	“Hospital Server has received a schedule request from a user with hash suffix: <hash_suffix> to book an appointment using TCP over port <hospital server TCP port number>.”
After forwarding the schedule request to the appointment server	“Hospital Server has sent the schedule request to the appointment server.”
After receiving the response from the appointment server	“Hospital Server has received the response from Appointment Server using UDP over <Hospital Server UDP port number>.”
After forwarding the response to the client	“The hospital server has sent the response to the client.”

Table 6. Hospital Server	
cancel request	
Upon receiving the cancellation request from the client	“Hospital Server has received a cancel request from user with hash suffix: <hash_suffix> to cancel their appointment using TCP over port <hospital server TCP port number>.”
After forwarding the cancel request to the Appointment Server	“The hospital server has sent the cancel request to the appointment server.”
After receiving the response from the appointment server	“Hospital Server has received the response from Appointment Server using UDP over port <hospital Server UDP port number>.”
After forwarding the response to the client	“The hospital server has sent the response to the client.”
view_appointment request (Patient)	
Upon receiving the patient’s view appointment request from the client	“Hospital server has received a view appointment request from a user with hash suffix <hash_suffix> to view their appointment details using TCP over port <Hospital Server TCP port number>.”
After forwarding the view appointment request to the Appointment Server	“Hospital Server has sent the view appointments request to the Appointment Server.”
After receiving the response from the Appointment Server	“Hospital Server has received the response from the appointment server using UDP over port <Hospital Server UDP port number>.”
After forwarding the response to the client	“The hospital server has sent the response to the client.”
view_appointments request (Doctor)	
Upon receiving the doctor’s view appointment request from the Client to view their appointment	“Hospital Server has received a view appointments request from <doctor_username> to view their schedule details using TCP over port <hospital server TCP port number>.”
After forwarding the doctor’s view appointment request to	“The hospital server has sent the view appointments request to the Appointment Server.”

Table 6. Hospital Server	
appointment server	
After receiving the response from Appointment Server	“Hospital server has received the response from the Appointment server using UDP over port <Hospital Server UDP port number>.”
After forwarding the response to the client	“The hospital server has sent the response to the client.”
prescribe request (Doctor Command)	
Upon receiving the prescription request from the client	“Hospital Server has received a prescription request from <doctor_username> for a user with hash suffix <hash_suffix> using TCP over port <hospital server TCP port number>.”
After fetching the patient’s illness information from Appointment Server	“Hospital Server has sent a request to fetch patients with hash suffix <hash_suffix> illness information to the Appointment Server.”
After receiving the response from Appointment server	“Hospital Server has received the illness response from the Appointment server using UDP over port <Hospital Server UDP port number>.”
Looking up treatment for an illness	“Acquiring treatment for <illness> from the database.”
Upon sending the treatment to the prescription server	“Hospital server has sent the prescription request to the prescription server to prescribe <treatment>.”
Upon receiving the response from the prescription server	“Hospital server has received the response from the prescription server using UDP over port <Hospital Server UDP port number>”
After forwarding the response to the client	“The hospital server has sent the response to the client.”
view_prescription request	
Upon receiving the prescription request from the client to view their prescription (Patient)	“Hospital Server has received a prescription request from a patient with hash suffix <hash_suffix> to view their prescription details using TCP over port <Hospital Server TCP port number>.”
Upon receiving the prescription	“Hospital Server has received a prescription request from

Table 6. Hospital Server	
request from the client (Doctor)	<doctor_username> to view a patient with hash suffix <hash_suffix> prescription details using TCP over port <Hospital Server TCP port number>.”
After forwarding the prescription request to the prescription server	“Hospital Server has sent the prescription request to the Prescription Server.”
After receiving the response from the prescription server	“Hospital server has received the response from the prescription server using UDP over port <hospital Server UDP port number>.”
After forwarding the response to the client	“Hospital server has sent the response to the client.”

The patient’s username should be displayed in plaintext within the client application; in all other servers, it must be represented by its hash suffix value.

Table 7. Client Server	
Event	**On-Screen Message**
Booting Up	“The client is up and running.”
Authentication Phase	
Failed login	“The credentials are incorrect. Please try again."
Upon sending the authentication request	“<username> sent an authentication request to the hospital server.”
Enter the correct credentials (**Patient**)	“<username> received the authentication result. Authentication successful. You have been granted patient access.”
Enter the correct credentials (**Doctor**)	“<username> received the authentication result. Authentication successful. You have been granted doctor access.”
help	
Asking for help commands	“Please enter the command:

Table 7. Client Server	
(**Patient**)	<lookup>, <lookup <doctor>>, <schedule <doctor> <start_time> <illness>>, <cancel>, <view_appointment>, <view_prescription>, <quit>”
Asking for help commands (**Doctor**)	“Please enter the command: <view_appointments>, <prescribe <patient> <frequency>>, <view_prescription>, <quit>”
lookup command	
Upon sending a lookup request	“<patient_username> sent a lookup request to the hospital server.”
Fetching Doctor list Response	“The client received the response from the hospital server using TCP over port <client port number>. The following doctors are available: <doctor1_username> <doctor2_username> …”
lookup <doctor name>	
Lookup availability request for this doctor	“Patient <patient_username> sent a lookup request to the hospital server for <doctor_username>.”
All time slots are available	“The client received the response from the hospital server using TCP over port <client port number>. All time blocks are available for <doctor_username>.”
Some time slots are available	“The client received the response from the Hospital Server using TCP over port <client port number>. <doctor_name> is available at times: <time block 1> <time block 2> …”
No time slots available	“The client received the response from the Hospital Server using TCP over port <client port number>.

Table 7. Client Server	
	<doctor_username> has no time slots available.”
schedule command	
Schedule request	“<patient_username> sent an appointment schedule request to the hospital server.”
Successfully scheduling	“The client received the response from the Hospital Server using TCP over port <client port number> An appointment has been successfully scheduled for patient <patient_username> with <doctor_username> at <start_time>.”
Failed scheduling (no time-blocks available)	“The client received the response from the hospital server using TCP over port <client port number> Unable to schedule an appointment with <doctor_username> at this time, as all time blocks have been taken up.”
Failed scheduling (some time-blocks available). Must enforce 9:00-17:00 (not inclusive) availability	“The client received the response from the hospital server using TCP over port <client port number> Unable to schedule an appointment with <doctor_username> at <selected time>. Other available time blocks are <time_slot 1> <time_slot 2> <time_slot 3> …”
cancel appointment	
Upon sending a cancellation request	“<patient_username> sent a cancellation request to the Hospital Server.”
No appointment to cancel	“The client received the response from the Hospital Server using TCP over port <client port number> You have no appointments available to cancel.”
Successfully cancelled appointment	“The client received the response from the Hospital Server using TCP over port <client port number> You have successfully cancelled your appointment with <doctor_username> at <time_slot>.”

Table 7. Client Server	
view_appointment command	
Request to view appointment	“<patient_username> sent a request to view their appointment to the Hospital Server.”
Appointment exists	The client received the response from the hospital server using TCP over port <client port number> You have an appointment scheduled with <doctor name> at <time_slot>.”
No appointment exists	“The client received the response from the hospital server using TCP over client port <client port number> You do not have an appointment today.”
view_prescription command	
Request to view prescription	“<patient_username> sent a request to view their prescription to the Hospital Server.”
Prescription does not exist	“The client received the response from the hospital server using TCP over port <client port number> You do not have a prescription to look up.”
Frequency value is None	“The client received the response from the hospital server using TCP over port <client port number> You were not prescribed any treatment by “<doctor_username> following your diagnosis.”
If a prescription exists	“The client received the response from the hospital server using TCP over port <client port number> You have been prescribed <treatment>, to be taken <frequency>, by <doctor_username>.”
view_appointments command (Doctor)	
Request to view appointments from the schedule	“<doctor_username> sent a request to view their scheduled appointments to the Hospital Server.”
All time slots are available	“The client received the response from the hospital server using TCP over port <client port number> You do not have any appointments scheduled.”
There is one or more	“The client received the response from the hospital server

Table 7. Client Server	
appointments scheduled	using TCP over port <client port number> <doctor_name> is scheduled at times: <time block 1> <time block 2> … “
prescribe <patient> <frequency> command (Doctor)	
Request to prescribe the patient Assumes - Patient exists in the appointment server - Treatment exists in hospital.txt	“<doctor_username> sent a request to the Hospital Server to prescribe <patient_username> following their diagnosis.”
After receiving the response from the hospital server	“The client received the response from the hospital server using TCP over port <client port number> You have successfully prescribed <patient_username> with <treatment>, to be taken <frequency>.”
view_prescription <patient_username> command	
Request to view prescription	“<doctor_username> sent a request to view <patient_username> prescription to the Hospital Server.”
Prescription does not exist	“The client received the response from the hospital server using TCP over port <client port number> <patient_username> does not have a prescription.”
If a prescription exists (Even with a frequency value of None)	“The client received the response from the hospital server using TCP over port <client port number> <patient_username> has been prescribed <treatment>, to be taken <frequency>, by <doctor_username_from_entry>.”
quit command	
quit	“You have successfully been logged out.” —Quit Program—

Assumptions:

1. You have to start the process in this order: hospital_server, authentication_server, appointment_server, prescription_server, and client. If you need to have more code files than the ones that are mentioned here, please use meaningful names and all small letters, and **mention them all in your README File.**
2. You are allowed to use blocks of code from Beej's socket programming tutorial (Beej's guide to network programming) in your project. However, you need to mark the copied part in your code.
3. When you run your code, if you get the message "port already in use" or "address already in use", please first check whether you have a zombie process (see the following).
4. If you do not have such zombie processes, or if you still get this message after terminating all zombie processes, try changing the static UDP or TCP port number corresponding to this error message (*port numbers below 1024 are reserved and must not be used*). If you have to change the port number, please mention it in your README file and provide the reasons.
5. You may create zombie processes while testing your code. Please make sure you kill them whenever you run your code. To see a list of all zombie processes, try this command:

   >>ps –aux | grep ee450

   Identify the zombie processes and their process number, and kill them by typing at the command line:

   >>kill -9 processNumber

Requirements:

1. Do not hardcode TCP or UDP port numbers; obtain them dynamically. Refer to Table 1 to see which ports are statically defined and which ones are dynamically assigned. Use the getsockname() function to retrieve the locally bound port number when ports are assigned dynamically.
2. The host name must be hardcoded as localhost (127.0.0.1) in all code.
3. The clients and backend servers should keep running and waiting for another request until the TAs terminate them by Ctrl-C or by the quit command. If they terminate before that, you will lose some points for it.
4. All the naming conventions and the on-screen messages must conform to the previously mentioned rules.
5. You are not allowed to pass any parameter, value, string, or character as a command-line argument except while running the client in Phase 1.
6. All on-screen messages must exactly conform to the project description. You should not add any additional on-screen messages. If you need to do so for debugging purposes, you must comment out all extra messages before submitting your project.
7. Please remember to close the socket and tear down the connection once you are done using that socket.

Programming Platform and Environment
All your submitted code MUST work well on the provided Ubuntu virtual machine. No additional package installation is allowed (except for the SHA256 files for C and C++ implementation). All submissions will only be graded on the provided Ubuntu. TAs/Graders won't make any updates or changes to the virtual machine. It's your responsibility to make sure your code works well on the provided Ubuntu. "It works well on my machine" is not an excuse. Your submission MUST have a Makefile. Please follow the requirements in the following "SubmissionRules" section environment setup: https://github.com/csci104/docker/

Programming Languages and Compilers
You must use only C/C++ on UNIX, as well as UNIX Socket programming commands and functions. Here are the pointers for Beej's Guide to C Programming and Network Programming (socket programming):

- http://www.beej.us/guide/bgnet/

(If you are new to socket programming, please study this tutorial carefully as soon as possible and before starting the project)

- http://www.beej.us/guide/bgc/

You can use a UNIX text editor like `emacs` to type your code and then use compilers such as `g++` (for C++) and `gcc` (for C) that are already installed on Ubuntu to compile your code. You must use the following commands and switches to compile your `.c` or `.cpp` files. It will make an executable by the name of "yourfileoutput":

```
gcc -o yourfileoutput yourfile.c
g++ -o yourfileoutput yourfile.cpp
```

Do NOT forget the mandatory naming conventions mentioned before!

Also, inside your code, you need to include these header files in addition to any other header files you think you may need:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <string.h>
#include <netdb.h>
#include <sys/types.h>
#include <netinet/in.h>
#include <sys/socket.h>
#include <arpa/inet.h>
```

#include <sys/wait.h>

For those using Python, implementations of UDP and TCP have been provided on Brightspace. You should be able to use Beej's guide to networking programming, as the structure of the commands remains similar.

To run your program, use python3 yourfile.py

Submission File and Folder Structure:
Your submission must have the following folder structure and the files:

- ee450_lastname_firstname.tar.gz
    - ee450_lastname_firstname
        - client
        - hospital_server
        - authentication_server
        - appointment_server
        - prescription_server
        - Makefile
        - readme.txt (or) readme.md
        - <Any additional header or implementation files YOU WROTE YOURSELF>

Please make sure that your name matches the one in the class list. The TAs will extract the tar.gz file, and will place all the input data files in the same directory as your source files. The executable files should also be generated in the same directory as your source files.

Do NOT include any input data files or executable files, as this will result in losing points.

Submission Rules

1. Along with your code files, include a README file and a Makefile. The Makefile requirements are listed below. The README file can be in any format (Markdown or txt). The only requirement is that the TAs should be able to open the file and read it in the studentVM (the VM your project will be graded in) without installing any additional software. In the README file, please include:
    a. Your Full Name as given in the class list
    b. Your Student ID
    c. What you have done in the assignment
    d. What your code files are and what each one of them does. (Please do not repeat the project description; just name your code files and briefly mention what they do.)
    e. The format of all the messages exchanged, e.g., username and password, are concatenated and delimited by a comma, etc.
    f. Any idiosyncrasies of your project. It should specify the conditions under which the project fails, if any.
    g. Reused Code: Did you use code from anywhere for your project? If not, say so. If so, state what functions and where they're from. (Also identify this with a comment in the source code.)
    h. The version of Ubuntu (only the Ubuntu versions that we provided to you) you are using

Reusing functions that are directly obtained from a source that does not belong to you (e.g., from the internet or generated by AI) with little to no modifications, is considered plagiarism—except for code from Beej's Guide or provided code. Whenever you are referring to an online resource, make sure to only look at the source, understand it, close it, and then write the code by yourself. The TAs will perform plagiarism checks on your code, so make sure to follow this step rigorously for every piece of code you submit.

****Submissions WITHOUT README and Makefile WILL NOT BE GRADED.****

Makefile tutorial:
https://www.cs.swarthmore.edu/~newhall/unixhelp/howto_makefiles.html

About the Makefile: The makefile should support the following functions:

TABLE 9. Process flow for testing

make all	Compiles all your files and creates executables
./hospital_server	Runs Hospital server
./authentication_server	Runs backend authentication server
./appointment_server	Runs backend appointment server
./prescription_server	Runs backend prescription server
./client <USERNAME> <PASSWORD>	Runs the client

TAs will first compile all code using **make all**. They will then open at least 6 different terminal windows. On 4 terminals will start the hospital, authentication, appointment, and prescription servers. On the remaining terminals, they will start the clients using "./client".

Remember that all programs should always be on once started. TAs will check the outputs for multiple input values. The terminals should display the messages shown in the On-screen Messages tables in this project write-up.

Here are the instructions:

1. On your VM, navigate to the directory containing all your project files. Remove all executable and other unnecessary files. **ONLY** include the required source code files, Makefile, and the README file.
    a. Now run the following two commands:
        >> tar cvf ee450_YourLastName_YourFirstName.tar *
        >> gzip ee450_YourLastName_YourFirstName.tar
    b. You will find a file named "***ee450_YourLastName_YourFirstName.tar.gz***" in the same directory. Please notice there is a space and a star(*) at the end of the first command. An example submission would be:
        First Name: John
        Last Name: Doe
    c. So the submission would be: ee450_Doe_John.tar.gz
    d. Any compressed format other than .tar.gz will NOT be graded!
2. Upload "ee450_YourLastName_YourFirstName.tar.gz" to the Digital Dropbox on the BrightSpace website
    a. (*BrightSpace -> EE450 -> Activities -> Assignments -> Project*).

3. After the file is uploaded to the dropbox, you must click on the “Submit” button to actually submit it. If you do not click on “Submit”, the file will not be submitted.
4. BrightSpace will keep only the last submission. Submission after the deadline is considered invalid.
5. BrightSpace will send you a “Dropbox submission receipt” to confirm your submission. Please check your emails to confirm your submission was successfully received. If you have not received a confirmation email, contact your TA if it always fails.
6. After receiving the confirmation email, please confirm your submission by downloading and compiling it on your machine. This is exactly what your designated TA would do, so please grade your own project from the perspective of the TA. If the outcome is not what you expected, try resubmitting and confirming again. We will only grade what you submitted, even though it’s corrupted.
7. Please take into account all possible technical issues, and expect heavy traffic on the BrightSpace website very close to the deadline, which may render your submission or even your access to BrightSpace unsuccessful.
8. Please DO NOT wait until the last 5 minutes to upload and submit, as technical issues may occur and you will miss the deadline. And a kind suggestion: if you still encounter bugs one hour before the deadline, please submit first to make sure you get points for your hard work!

There is absolutely zero tolerance for late submissions! Do NOT assume that there will be a late submission penalty or a grace period. If you submit your project late (no matter what the reason or excuse, or even technical issues), you simply receive a ZERO for the project.

Grading Criteria

Notice: We will only grade what is already done by the program instead of what will be done. The grading criteria are subject to change.

Your project grade will depend on the following:

1. Correct functionality, i.e., how well your programs fulfill the requirements of the assignment, especially the communications through UDP and TCP sockets.
2. Inline comments in your code. This is important because it will help you understand what you have done.
3. Whether your programs work as you say they would in the README file.
4. Whether your programs print out the appropriate error messages and results.
5. Your code will only be tested on a fresh copy of the provided Virtual Machine (https://github.com/csci104/docker/). If your programs are not compiled or executed on these VMs, you will receive only the minimum points as described below. Be careful if you are going to use other environments!!! Do not update or upgrade the provided VM as well!!!
6. If your submitted code does not even compile, you will receive 5 out of 100 for the project.

7. If your submitted codes compile with make but, when executed, produce runtime errors without performing any tasks in the project, you will receive 10 out of 100.
8. The minimum points for compiled and executable codes is 15 out of 100.
9. If your code does not correctly assign the TCP or UDP port numbers (in any phase), you will lose points each.
10. We will use similar test cases to test all the programs. These test cases cover all situations, including edge cases, as described in the on-screen messages section.
11. There are no points for the effort or the time you spend working on the project or reading the tutorial. If you spend about 2 weeks on this project and it doesn't even compile, you will receive only 5 out of 100.
12. You must discuss all project-related issues on the Piazza Discussion Forum. We will give extra points to those who actively help others out by answering questions on Piazza. (If you want to earn the extra credits, do remember to leave your names visible to instructors when answering questions on Piazza.)
13. Your code will not be altered in any way for grading purposes and however, it will be tested with different inputs. Your TA/Grader runs your project as is, according to the project description and your README file, and then checks whether it works correctly or not. If your README is not consistent with the description, we will follow the description.

Cautionary Words:

1. Start on this project early!!!

2. In view of what is a recurring complaint near the end of a project, we want to make it clear that the target platform on which the project is supposed to run is *the provided **Ubuntu (20.04)***. It is strongly recommended that students develop their code on this virtual machine. In case students wish to develop their programs on their personal machines, possibly running other operating systems, they are expected to deal with technical and incompatibility issues (on their own) to ensure that the final project compiles and runs on the requested virtual machine. If you do development on your own machine, please allow at least 3 days for it to work on Ubuntu. It might take much longer than you expect because of some incompatibility issues.

3. You may create zombie processes while testing your code. Please make sure you kill them whenever you run your code. To see a list of all zombie processes, try this command:

   >>ps –aux | grep ee450

   Identify the zombie processes and their process number, and kill them by typing at the command-line: >>kill -9 processnumber